

Software Requirement Specification

→ it is a structured document that systematically organise requirements (gathered req.) in form of document.

→ the SRS document describes

- (1) what operations the SW must support
- (2) under what conditions
- (3) with what constraints

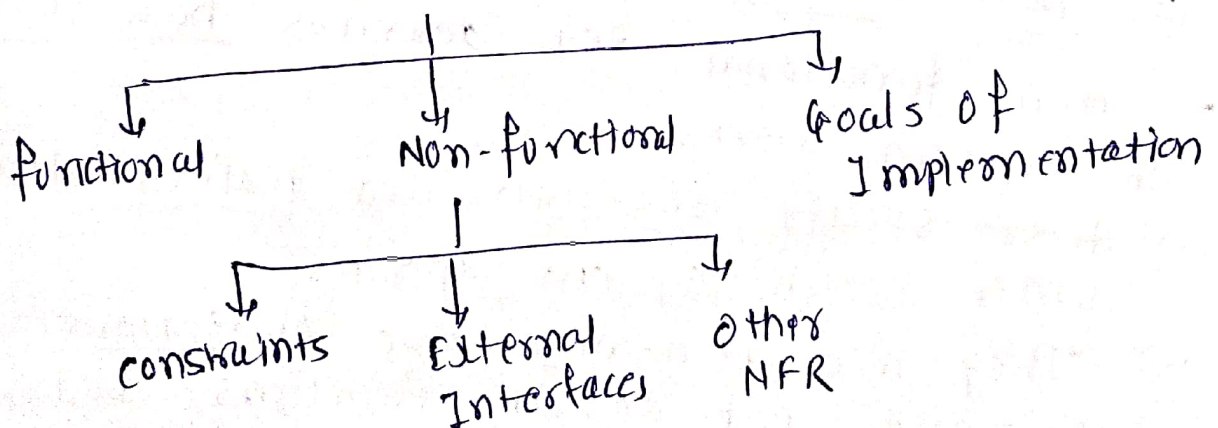
→ SRS documents what the systems should do not how it will be built.

→ where SRS required in SW Engineering?

- (1) customer needs (Requirements)
- (2) Requirement Gathering
- (3) Requirement Analysis
- (4) Requirement specification (SRS document)

- (5) Design
- (6) Implementation
- (7) Testing

* categories of customer requirements



① How to classify different types of requirements?

Step 1: Ask - is it a function?

→ Does it transform input data to output data?

if yes → Functional Req.

Step 2: if not a function,

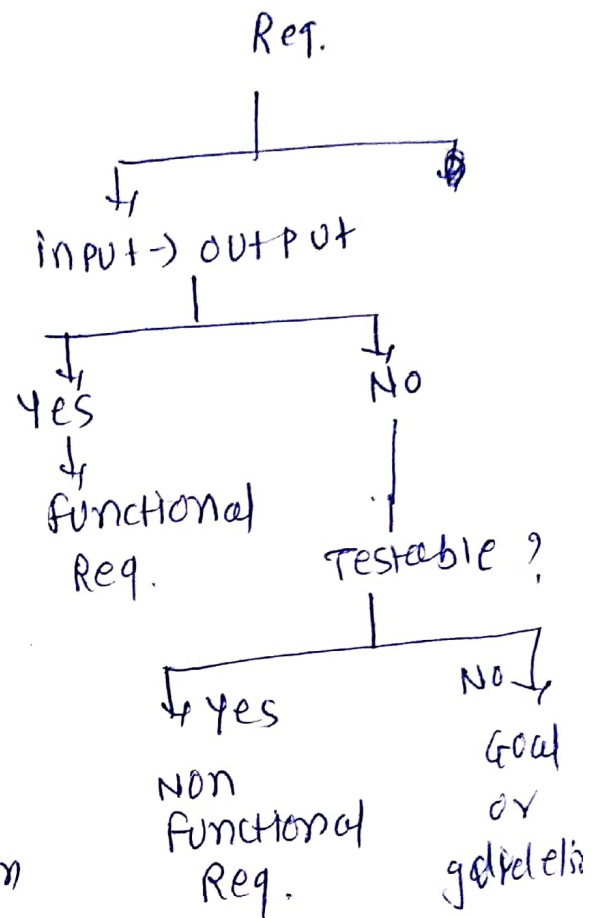
→ ask is it testable at acceptance?

if yes → Non-functional Requirement

Step 3: if not testable, then

→ Is it only suggestion or future desire?

if yes → Goal of Implementation



example with comparison

<u>category</u>	<u>key question</u>	<u>tested?</u>	<u>example</u>
① Functional	I/P → O/P?	Yes	search book
② Non-functional	Quality / constraint?	Yes	search < 2 sec
③ Goals of Impl.	suggestion only?	No	future e-book support

As per IEEE 830 the important categories of user requirements are following.

1. Functional Requirements

- These specify what the system must do.
- They describe the services/ functions provided by the system.
- They can be viewed as a mathematical function which describes the transformation of input $\rightarrow$ o/p.

$$f: I \rightarrow O$$

Input $\rightarrow$ processing $\rightarrow$ output.

example

BOOK name $\rightarrow$ search logic $\rightarrow$ availability (list of books)

- they form the basis of testing and design.

examples (all functions performed by all user comes in this category)

1. Search a book by title or author
2. Issue a book to a member
3. Renew a borrowed book.
4. Return book.

② Non-Functional Requirements

→ Non-Functional Req. describes how well the system works.

- ~~these~~ specify constraints and quality attributes under which system operates.

- They are Non-negotiable obligations.

- They do not describe functions, but how well or under what conditions functions must work.

- They are verifiable during acceptance testing.

③ Goals of Implementation

- These are desirable but not mandatory features
- general suggestions
- They are not tested during acceptance.
- They guide developers but are not binding.
- Failure to achieve them does not mean rejection.

ex → The design should support easy UI design later.
→ The system should be easily extensible for e-books in future.

④ Functional Requirement

How to identify functions of the system?

(*) What is function?

- A unit of system behavior that transforms user input into meaningful output to achieve a specific user objective.

Functional Requirement



High-level Functional Req.



Sub-Functional Requirements

⇒ High level functional req. & sub-requirements

- High level functional is one through which a user gets some meaningful piece of work done.
- High level req. involves accepting some data from user interface, transforming in required response and displaying system response
- It includes a series of interactions between system and one or more users

- each user input and corresponding system action may be considered as sub-requirement.
- High level requirement can consists of several sub requirements

Example

R1: Issue BOOK (High-level functional req.)
subfunctional Req.

- R1.1 : validate member Id
- R1.2 : check Book availability
- R1.3 : verify borrowing limit
- R1.4 : Record Issue transaction
- R1.5 : Update book status
- R1.6 : Print issue receipt

Note → High level requirement is achieved only when all these sub-functions are executed.

Note → this can't be FR, it is sub FR.

④ Functional Requirements involve Interaction Scenarios

- A functional req. consists of a sequence of user actions and system responses rather than a single input-output step.
- SW systems are interactive.
- users and systems communicate in multiple steps.
- each step influence the next step.
- hence functional req. has multiple scenarios.

General Interaction Pattern

User Action → System Response → User action
 ↓
 System Response
 ↓
 Goal achieved.

Example: Issue BOOK.

Enter BOOK NAME

↓
display book details

↓
User request issue

↓ → check book availability

Issue BOOK / Display error message.

→ All these step together define one functional requirement Issue BOOK.

⇒ IS it possible to determine all INPUT & OUTPUT data precisely during req. specifcn.

→ mostly no.

Why?

- ① Precision is desirable but not always possible
- ② difficulty increases without a working model
- ③ High level req. use High level data terms

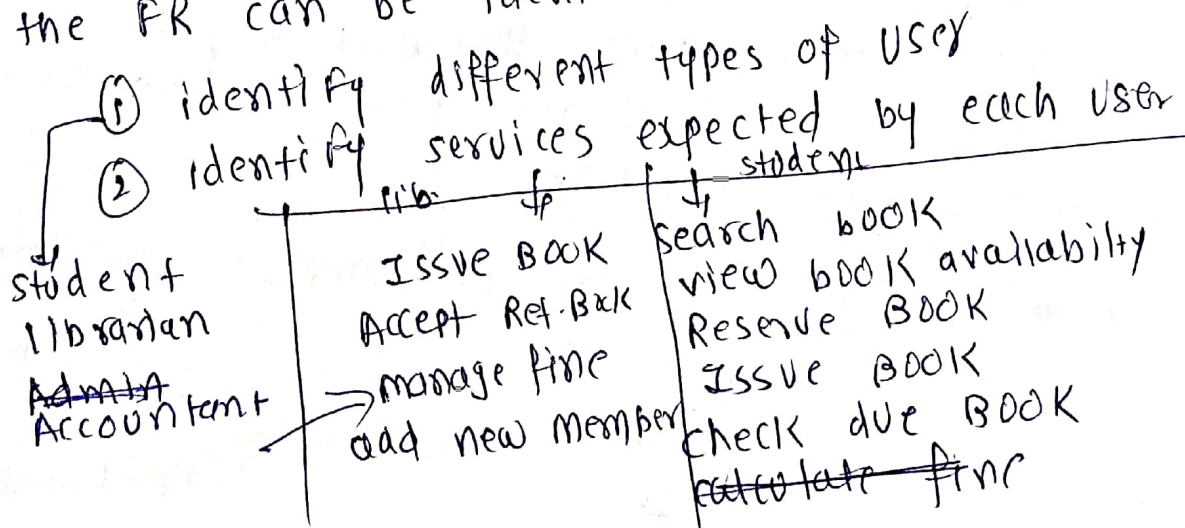
ex Issue BOOK

(enter BOOK ID, mem ID ...)

- ④ Data is often entered in stages.
- ⑤ Precision is achieved through decomposition in sub functions.

④ How to identify functional requirements

→ the FR can be identified in two ways



ex successful issue
BOOK not available
user limit exceed

④ How to document functional Requirements?

→ two accepted ways, both are correct

① user-wise

② function wise

- choose user wise for user-centric system
- function wise for process centric system.
- if many functions are common between them then choose function wise.

1. User-wise

Student

- ↳ search BOOK
- ↳ ~~via~~ Reserve BOOK

lib.

- ↳ add BOOK
- ↳ Issue BOOK
- ↳ Return BOOK

2. function-wise

BOOK management

- ↳ Add BOOK
- ↳ search BOOK
- ↳ issue BOOK
- ↳ Return BOOK

⑤ Traceability in Requirement Engg.

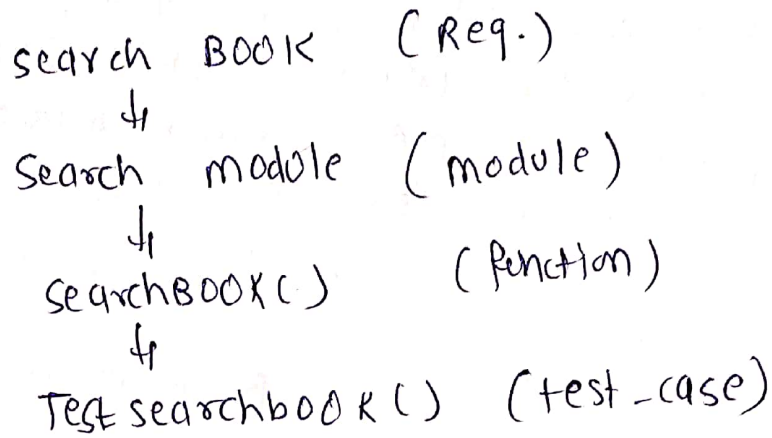
- it refers to the ability to follow a requirement throughout the SW lifecycle
- Bi-directional traceability must support
req. → design → code - test
code / test → design → req.

How it can be achieved?

- ↳ unique and consistent req. numbering
High level req :- R1, R2, R3
sub-function req. → R1.1, R1.2, R2.1 ...
- clear referencing documents.
- easy mapping

- Traceability can be directly linked to
- design modules
 - code functions
 - Test cases.

example



⇒ categories of Non-functional Requirements.

① Design and Implementation constraints

- restrictions that limit the choices available to developers during design and implementation
- they specify what must be followed or used
- restrict technology, platform, standards or policies.
- They are mandatory.

Example:- programming language, database to be used, OS, Regulatory policies, security standard.

Example:-

- ① The library system must use a free & open-source DBMS.
- ② The system must run on both windows & linux.
- ③ The system must comply with IT security policies.

② External Interface required.

- These specify how the SW interacts with external entities.

Types of external interface Requirements

1. User Interface → define how user interact with the system.

Example:- ① web based interface

② mobile app

③ command line app

- the Book availability shall be accessible through a web browser.

2. Hardware Interface → Interaction of SW with Physical components.

Example:- Barcode Scanner, printer

Example:- The system shall interface barcode scanner for book issue.

3. Software Interface requirements

→ Interaction with external software systems

ex Database, Payment gateway, ERP systems

ex → the system shall integrate payment gateway to collect fine through digital system.

4. Communication Interface

→ defines communication protocol & data exchange.

ex HTTP / HTTPS, REST API, TCP sockets, Message queues.

③ Other Non-functional Requirements

→ these capture quality attributes that affect system behaviour but are not constraints or interfaces.

→ common attributes are following

1. Performance

→ specify the speed and efficiency with which the system must respond to user requests.

ex 1. Book search result must display within 2 seconds
2. System must handle search req. for 500 concurrent users.

2. Reliability

→ system's ability to operate without failure for a specific period.

→ how often the system fails.

ex

- ① library system shall operate continuously for 24 HR.
- ② the system shall not lose issues or returned books records during normal operation.
- ③ In case of failure the system shall store the last consistent state from DB.

3. Availability

→ it specifies when and for how long system is operational & accessible.

ex 1. the system must be available during working hours.

2. BOOK search must be available 24x7.

3. Scheduled maintenance shall not exceed 2 HRS per month.

4. Security

→ defines protection against unauthorized access, misuse or loss of data.

ex 1. only authenticated members can issue books

2. students must login using valid credentials for renew book.

3. The system shall maintain access logs for issue and return transactions

5. Accuracy

→ Specify correctness & precision of data & o/p.

ex 1. system shall correctly update no of books after issue or return operation performed.

2. Fine calculation should be accurate to RS.

6. scalability

→ specify system's ability to handle growth in users, data & workload.

ex system should support 10,000 to 1,00,000 books without degradation.

→ must support 1000 concurrent user in future.

→ must allow addition of new branches without redesigning DB.